

10주차 세션 요약

토큰화 개요

자연어와 자연어처리

- 자연어(자연 언어): 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리(NLP): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

토큰화

- 자연어 처리 모델을 만들기 위해 말뭉치를 토큰으로 나누어야 함
- 토큰나이저를 구축하는 다양한 방법 존재
 - 공백 분할, 정규표현식 적용, 어휘 사전 적용, 머신러닝 활용
 - 토큰나이저의 구축 방법은 시스템의 성능/처리 결과에 큰 영향을 주기도 함

단어 및 글자 토큰화

단어 토큰화

- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업으로 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용됨
- 가장 일반적인 토큰화 방법

글자 토큰화

- 글자 단위로 문장을 나누는 방식
- 비교적 작은 단어 사전을 구축할 수 있다는 장점
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용

자모 라이브러리를 활용한 토큰화

- 비교적 적은 크기의 단어 사전 구축 가능, 접사와 문장 부호의 의미 학습 가능
- OOV 획기적으로 줄일 수 있음
- 개별 토큰의 의미가 없으므로 자연어 모델이 각 토큰의 의미 조합해 결과를 도출해야 하는 단점

형태소 토큰화

- 형태소 단위로 나누는 토큰화 방법으로, 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류
- 교착어인 언어에서 중요하게 수행됨 (한국어)

형태소 어휘 사전

- 각 단어의 형태소 정보를 포함하는 사전으로 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등의 정보를 함께 제공
- 품사 태깅: 텍스트 데이터를 형태소 분석하여 해당하는 품사를 태깅하는 작업
→ 이를 통해 문맥 고려 가능

형태소 토큰화 라이브러리 KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
- Okt, 꼬꼬마(Kkma), 코모란(Komoran) 등의 다양한 형태소 분석기 지원

형태소 토큰화 라이브러리 NLTK

- 자연어 처리를 위해 개발된 라이브러리
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공
- 주로 영어 자연어 처리를 위해 개발
- 토큰화, 품사 태깅 작업을 위해 패키지나 모델을 다운로드 해야 함
 - Punkt 모델, Averaged Perception Tagger 모델

형태소 토큰화 라이브러리 spaCy

- 빠른 속도와 높은 정확도를 목표로 하는 머신 러닝 기반의 자연어 처리 라이브러리
- NLTK: 학습 목적 ↔ spaCy: 효율적인 처리 속도와 높은 정확도

하위 단어 토큰화

기존 형태소 분석기의 문제점

- OOV(Out-of-Vocabulary): 신규어/신조어/도메인 특화 용어 처리 취약
- 형태소 경계 의존: 띄어쓰기/품사 태깅 오류 전파
- 언어/도메인 편향: 특정 코퍼스 · 사전 설계에 과적합
- 유연성 부족: 다국어/도메인 확장 시 재설계/재학습 비용 큼

하위 단어 토큰화

- 너무 미세(문자)도, 너무 거칠(단어)지도 않게 중간 단위로 분해
- 장점: OOV 감소, 어휘축소(메모리 · 연산 절감), 형태 변형(어미/접사) 대응
- 원칙: 자주 함께 나타나는 문자열을 묶어 서브워드 단위로 모델링

BPE(Byte Pair Encoding)

- 문자 단위 시작 → 가장 자주 출현하는 인접 쌍 병합 반복
 - 병합 규칙(merges) + 어휘(vocab)
 - 빈도 기반, 간단 · 효율적, 신어 일반화 강함

SentencePiece & Korpora 라이브러리

SentencePiece

- 구글에서 개발한 오픈소스 하위 단어 토크나이저 라이브러리
- 바이트 페어 인코딩과 유사한 알고리즘
- 사용자가 직접 설정할 수 있는 하이퍼파라미터 제공을 통해 세밀한 토크나이징 기능 제공
- 데이터 정규화 · 특수 토큰 처리 포함, 재현성 우수의 장점

Korpora(코포라)

- 한국어 공개 코퍼스 수집 패키지, 실습용 말뭉치 쉽게 확보 가능

토크나이저 학습의 공통 프로세스

1. 데이터 수집: 도메인 대표성/크기/클린
2. 정규화: lowercasing, NFD/NFKC, 특수문자 규칙
3. 프리토큰화: 공백/ByteLevel/Metaspace 등
4. 모델 학습: BPE/WordPiece/Unigram, vocab 크기 결정
5. 포스트프로세싱: [CLS]/[SEP] 삽입 등 템플릿 처리
6. 평가: 토큰 길이, 커버리지, OOV, Perplexity 대리 지표
7. 저장/배포: 버전 고정, 재현성/호환성 체크

실습1: SentencePiece + Korpora(BPE)

1. 청원 말뭉치 로드, 학습 파일 생성
 - load("korean_petitions") → 청원 말뭉치 사용.
 - get_all_texts() → 한 줄 한 문장 형태 저장(학습 입력 준비)

2. SentencePieceTrainer로 BPE 모델 학습
model_type=bpe : 교재의 BPE 설정
→ .model(모델), .vocab(어휘) 생성
 3. SentencePieceProcessor로 encode/decode
→ 토큰, ID, 디코딩 확인
 4. SentencePiece 어휘 사전 확인 - vocab 조회 & 특수 토큰 확인
→ <unk>, <s>, </s>와 vocab 크기 확인
-

WordPiece

- 확률기반 최대우도 기준으로 어휘를 선정
- BPE: 빈도 기반 병합 ↔ WordPiece: 언어모델 우도/점수 사용

Hugging Face tokenizers 라이브러리

- BPE/WordPiece/ Unigram / ByteLevel 등 지원
- 정규화/프리토큰화/포스트프로세싱 모듈화
- 불필요 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수문자 처리 등을 제공
- 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효과적으로 처리
- 대용량 코퍼스에서도 빠른 학습, transformers와 호환

WordPiece 토크나이저 모델 학습 과정

1. 코퍼스 수집 · 정제(Korpora/사내 텍스트)
2. 정규화 규칙 확정(NFKC, 숫자/기호 규칙)
3. 프리토큰화(Whitespace/ByteLevel)
4. WordPiece 학습(어휘 크기, ## 접두사)
5. 포스트프로세싱([CLS]/[SEP] 템플릿)
6. 품질평가(토큰 길이/커버리지/OOV율)
7. 버전관리 및 배포(JSON 저장)

실습2: tokenizers(WordPiece)

1. tokenizers로 WordPiece 구성 · 학습 · 저장
2. WordPiece 로드 & 디코더 설정
→ 토큰화 결과를 자연스럽게 복원

3. WordPiece 인코딩/배치 인코딩/디코딩 실습

decode()로 문장 복원 확인

→토큰/ID/복원 결과 확인

정규화: NFD()(유니코드 분해), Lowercase(), 사전 토큰화: Whitespace() 사용함

임베딩

- 컴퓨터는 텍스트 자체를 이해할 수 없기 때문에 텍스트 벡터화 과정이 필요

텍스트 벡터화(Text Vectorization)

- 텍스트를 숫자로 변환하는 과정
- 원-핫 인코딩(One-Hot Encoding)
 - 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑 후 해당 색인 위치를 1로 표시, 나머지 위치는 모두 0으로 표시하는 방식
- 빈도 벡터화(Count Vectorization)
 - 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식
 - 장점: 단어나 문장을 벡터 형태로 변환하기 쉽다
 - 단점: 벡터의 희소성(Sparsity)이 크다
컴퓨팅 비용의 증가, 차원의 저주
벡터가 텍스트의 의미를 반영하지 못한다

워드 임베딩

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법
- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론
- 대표 기법: Word2Vec, fastText
- 단점: 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다
→ 동적 임베딩(Dynamic Embedding) 기법 사용

언어모델(Language Model)

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행
- 자동 번역, 음성 인식, 텍스트 요약 등 자연어 처리 분야에서 활용
- 단점: 주어진 문장 뒤에 나올 수 있는 문장이 다양하기 때문에 완성된 문장 단위로 확률을 계산하는 것은 어려움

→ 자기회귀 언어 모델: 하나의 토큰 단위로 예측

자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
- 조건부 확률의 연쇄법칙(Chain rule for conditional probability) 적용
→ 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측
- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
→ 토큰 분류 문제로 정의 가능
- 모델의 출력값이 모델의 입력값으로 사용됨
- 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측

통계적 언어 모델 (Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
- 시퀀스에 대한 확률 분포를 추정 -> 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측
- 마르코프 체인(Markov Chain)을 이용해 구현됨
→ 데이터 희소성(Data sparsity) 문제 발생

N-gram

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단어로 취급 → 특정 단어 시퀀스가 등장할 확률 추정
- 유니그램($N=1$), 바이그램($N=2$), 트라이그램($N=3$), N-gram($N \geq 4$)이라고 부름

TF-IDF

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법
- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치
- Bow에 가중치를 부여하는 방법

BoW (Bag-of-Words)

- 문서나 문장을 단어의 집합으로 표현하는 방법
- 문서/문장에 등장하는 단어의 중복을 허용해 빈도를 기록

단어 빈도 (Term Frequency, TF)

- 문서 내에서 특정 단어의 빈도수를 나타내는 값
- 해당 단어의 상대적인 중요도를 측정하는 데 사용
- 높은 TF → 특정 문서에서 중요한 역할 / 전문 용어나 관용어

문서 빈도 (Document Frequency, DF)

- 한 단어가 얼마나 많은 문서에 나타나는지
- 높은 DF → 일반적으로 널리 사용되는 단어 / 중요도 낮음
- 낮은 DF → 특정 문맥에서만 사용되는 단어 / 중요도 높음

역문서 빈도(Inverse DF, IDF)

- 문서 내에서 특정 단어가 얼마나 중요한지
- 전체 문서 수를 문서 빈도로 나눈 다음 로그 취한 값
- 높은 IDF → 일반적이고 중요하지 않은 단어

TF-IDF

- 문서 빈도와 역문서 빈도를 곱한 값
- 관사/관용어 등의 가중치 낮아짐

실습: TF-IDF 벡터화

TF-IDF 클래스의 주요 매개변수

- 입력값(input): 입력될 데이터의 형태
- 인코딩(encoding): 바이트/파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
- 소문자 변환(lowercase): 입력받은 데이터를 소문자로 변환할지 여부
- 불용어(stop_words): 분석에 도움이 되지 않는 의미 없는 단어
- N-gram 범위(ngram_range): 사용할 N-gram의 범위
- 최대값 문서 빈도(max_df): 전체 문서 중 일정 횟수 이상 등장하는 단어는 불용어
- 최소값 문서 빈도(min_df): 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리
- 단어사전(vocabulary): 미리 구축한 단어 사전
- IDF 분모처리(smooth_idf): IDF 계산 시 분모에 1 더함

TF-IDF 객체 생성 및 학습 과정

1. 객체 불러오기
TfidfVectorizer 클래스로 tfidf_vectorizer 객체 생성
2. fit() 메서드
fit() 메서드를 통해 말뭉치 학습, 모델이 단어 사전을 구축
3. transform() 메서드
학습된 객체를 사용해 데이터를 TF-IDF 벡터로 변환
4. fit_transform() 메서드
학습과 변환을 한 번에 수행

TF-IDF의 한계점

- 문맥과 단어의 순서를 고려하지 않음
→ 문장 생성이나 문장 순서가 중요한 작업에 부적합
- 단어의 의미를 반영하지 않음
→ TF-IDF 벡터는 중요도만 나타내며, 단어 간 의미적 유사성을 표현하지 못함